



Graphic Era University, Dehradun

Department of Computer Science & Engineering

Project Based Learning (PBL) – Phase I Synopsis

DYNAMIC SMART CITY ENGINE

A Dynamic Incident Prioritization and Resource Coordination System for Urban Services

Course: Data Structure and Programming in C++ | Semester: 3rd

Team ID	DSCPP-III-2026-T374	Semester	3rd
Course	Data Structure and Programming in C++	Mentor	Mr. Kireet Joshi
Name	Roll No.	Section	Role
Pavani Jain	260222072	DS1	Team Leader
Karan Babbar	2029058	SEC-B	Team Member
Manan Kumar	2028450	DS2	Team Member

Mentor email: kireetjoshi@geu.ac.in

1. Motivation

Dehradun has been growing fast over the last few years, and under the Smart City proposal, a number of projects were planned for things like traffic and transport management, parking, surveillance, environmental monitoring, solid waste management and a command-and-control setup for the city. While reading up on this for our project, we came across the CAG's audit of the Smart Cities Mission in Dehradun. It pointed out that several end devices had operational availability issues, and that the sensor-based solid waste management module wasn't really being used the way it was supposed to.

That is honestly what got us thinking about the direction of this project. Installing sensors and cameras across a city sounds impressive, but it doesn't automatically make city services run better. Someone, or something, still has to decide which problem needs attention first when multiple things go wrong at the same time. A traffic signal failure at a busy junction is usually more urgent than a routine, low-severity complaint sitting in a queue somewhere. But at the same time, a complaint that has been pending for a long time can become urgent on its own, just because of how long it has waited.

So the idea we settled on is a **Dynamic Smart City Engine** – basically a decision-support system rather than one more data-collection layer. It takes in simulated city incidents, works out how severe and how urgent each one is, checks what resources are actually free, and then suggests a response. It also keeps track of an incident all the way from when it's reported to when it's finally closed. We're not claiming this solves every problem a smart city has; the goal is much smaller than that. We just want to build a realistic engine that shows how this kind of prioritization and coordination could actually work in practice.

2. State of the Art / Current Solution

Most existing Smart City systems depend on sensors, CCTV, GPS, traffic devices and an integrated command-and-control centre to gather information from the different city services. Dehradun's own Smart City plan follows the same pattern – traffic management, surveillance, environmental monitoring, e-governance and solid-waste components, all tied to a central command-and-control centre.

The issue is that these systems are quite good at collecting and displaying information, but that's not the same as acting on it. Administrators still have to work out, more or less manually, which issue is more urgent, whether a resource is actually available right now, and how the whole response should be coordinated. The CAG audit we mentioned earlier backs this up too, since it specifically flagged problems with device availability and with the waste management module not being used properly in Dehradun's implementation.

This is the gap our project is trying to fill – not the sensors or the cameras, but the decision-making layer sitting on top of them. Instead of just putting numbers on a dashboard, the Dynamic Smart City Engine actually processes incidents using priority rules and the right data structures for each job. It ranks incidents, keeps track of resources and infrastructure status, and uses a graph-based model of the city for anything route-related. Put simply, it's a simplified prototype that connects raw incident information to an actual, priority-based action, which is the part we felt was missing.

3. Project Goals and Milestones

Our main goal for this project is to build a working prototype that can dynamically manage and prioritize simulated city incidents. Along the way, we also want to show, properly and not just superficially, how Data Structures and OOP concepts can be applied together to solve a problem that actually resembles something real.

Main Goals

- Develop a system to record different types of city incidents.
- Calculate the priority of each incident dynamically.
- Monitor the operational status of selected city infrastructure.
- Maintain a list of available resources and recommend suitable allocation.
- Represent selected city locations using a graph.
- Find efficient response routes between resources and incident locations.

- Track the progress of an incident from reporting to resolution.
- Apply the compulsory Data Structures and C++ OOP concepts in meaningful modules, not as an afterthought.

Initial Milestones

- Finalize requirements and problem statement.
- Design the classes and data structures.
- Create the basic incident-management module.
- Implement priority calculation.
- Implement resource management.
- Create the city graph and route module.
- Add searching, sorting, hashing and tree-based storage.
- Implement file handling and persistent reports.
- Test the complete prototype using simulated Dehradun-inspired scenarios.

4. Project Approach

We're planning to build this as a C++-based simulation and decision-support system. The city itself will be represented through a set of selected zones, locations, resources and infrastructure entities – we don't intend to model the whole of Dehradun, just enough of it to make the scenarios believable. Each incident will carry information such as its type, location, severity, number of people affected, time it was reported, and its current status.

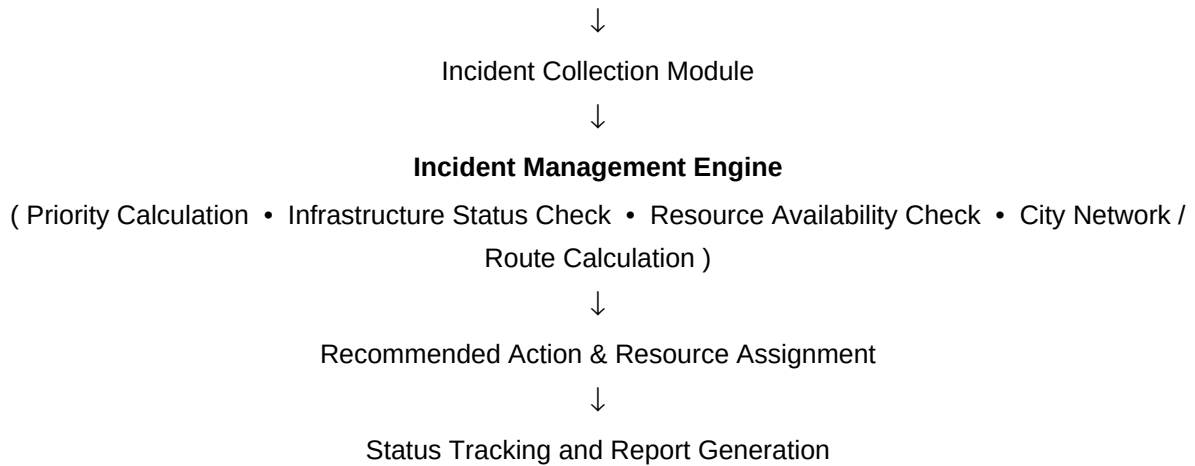
Different modules will use whatever data structure actually fits the job, instead of forcing one structure everywhere just to tick a box. Linked lists will hold the incident records since they keep growing dynamically, queues will manage incidents that are waiting to be processed, graphs will represent the city network, and hashing will give us quick access to resources or incidents through their unique IDs. Trees will handle organized searching, and sorting will be used to rank incidents by priority.

On the OOP side, we're using an abstract **CityEvent** base class for the properties every event has in common. Classes like **TrafficEvent**, **WasteEvent**, **InfrastructureEvent** and **EmergencyEvent** will inherit from it, and polymorphism lets each event type work out its own priority based on its own rules, rather than one big if-else block trying to handle everything.

For this first version, we're going with simulated input rather than live government or sensor data. Mainly because getting access to real Smart City data isn't really feasible for a student project in this timeframe, and simulated data still lets us build and test fairly realistic scenarios. It's enough to show how incidents get prioritized dynamically and how the available resources get coordinated, which is really the point we're trying to prove.

5. System Architecture

Simulated City Data / User Input



Main Components

- Incident Management Module
- Priority Engine
- Infrastructure Monitor
- Resource Management Module
- City Graph and Route Module
- Data Storage and File Module
- Report / Output Module

6. Project Outcome / Deliverables

The main deliverable is a working C++ prototype – the Dynamic Smart City Engine. It will simulate different urban incidents and show how they get prioritized dynamically, based on factors like severity, impact and how long an incident has been sitting unresolved.

The system will have an incident-management module, infrastructure status monitoring, resource management, and a graph-based model of the city network. A user should be able to add and search incidents, see them ranked by priority, assign whatever resources are available, and track each incident's progress from the moment it's reported to when it's actually resolved.

Through building this, we'll also be demonstrating the compulsory Data Structures concepts – linked lists, queues, stacks, trees, graphs, hashing, searching and sorting – along with the required C++ OOP concepts like classes, inheritance, polymorphism, abstract classes, virtual functions, STL, exception handling and file handling. We're trying to fit these in naturally where they make sense for the module, not just cram every concept into one screen for the sake of the checklist.

For the final submission we'll be handing in the source code, our GitHub repository, technical documentation, the system architecture, our test scenarios, and the Phase-I presentation.

7. Assumptions

Since this is a prototype, it doesn't connect to any real government system or live sensor feed. All the city incidents and infrastructure information will be simulated to begin with, through predefined scenarios and manual input. The Dehradun locations we select will be represented as nodes in a simplified graph, so the routes we calculate are conceptual and not actual real-time navigation results – we want to be upfront about that. Resource availability and incident priority are also based on rules we're defining ourselves for this project, not on any external data source. Overall, the aim here is to demonstrate the decision-making and coordination concepts in a realistic, technically feasible way, within what's reasonable for a student project of this scope.

References: Dehradun Smart City project information (Smart City Mission proposal documents); Comptroller and Auditor General of India, audit report on the Smart Cities Mission, Dehradun.